

Python Tutorial

A Complete Guide

19 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/python/>

Getting Started with Python

What is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasizes code readability with significant indentation. Python is dynamically-typed and garbage-collected, supporting multiple programming paradigms including procedural, object-oriented, and functional programming.

A Brief History

Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0 was released in 2000, and Python 3.0 (breaking backward compatibility) arrived in 2008. As of 2025, Python 3.x is the only actively developed version, with Python 2 officially retired on January 1, 2020.

Why Learn Python?

Python is one of the most popular programming languages in the world. It is used in web development (Django, Flask), data science (NumPy, pandas), machine learning (TensorFlow, PyTorch), automation, scripting, and more. Its simple syntax makes it an excellent first language for beginners.

Key Features

- **Readable & Expressiveness** — Clean syntax with minimal boilerplate.
- **Interpreted** — Code runs immediately without a separate compilation step.
- **Dynamically Typed** — Variables do not need explicit type declarations.
- **Cross-Platform** — Runs on Windows, macOS, Linux, and more.

- **Rich Standard Library** — Batteries included for file I/O, networking, threading, and more.

Installing Python

Visit python.org/downloads and download the latest version for your operating system. During installation on Windows, make sure to check "Add Python to PATH".

Verify the Installation

Open a terminal (command prompt, PowerShell, or terminal) and run:

```
python --version
```

You should see output similar to `Python 3.13.x`.

Virtual Environments (venv)

A virtual environment is an isolated Python environment that lets you install packages specific to a project without interfering with your system-wide Python installation.

```
# Create a virtual environment
python -m venv myenv

# Activate it (Windows)
myenv\Scripts\activate

# Activate it (macOS/Linux)
source myenv/bin/activate

# Deactivate when done
deactivate
```

Your First Program

The quintessential first program in any language is printing "Hello, World!":

```
print("Hello, World!")
```

Save this in a file called `hello.py` and run it with:

```
python hello.py
```

Interactive Interpreter

Python also provides an interactive REPL (Read-Eval-Print Loop). Just type `python` in your terminal and start writing code line by line:

```
>>> print("Hello from the REPL!")
Hello from the REPL!
>>> 2 + 2
4
```

Interpreter vs Script

You can run Python in two ways:

- **Interactive interpreter** — Great for experimenting and testing small snippets.
- **Script mode** — Running a `.py` file containing multiple lines of code. This is how real programs are executed.

Tip: Use the interactive interpreter to test ideas quickly, but always save your work as scripts.

IDE and Editor Setup

You can write Python in any text editor, but a good IDE boosts productivity.

PyCharm

PyCharm (by JetBrains) is a full-featured Python IDE. The free Community Edition is sufficient for learning. It offers intelligent code completion, debugging, and integrated virtual environment management.

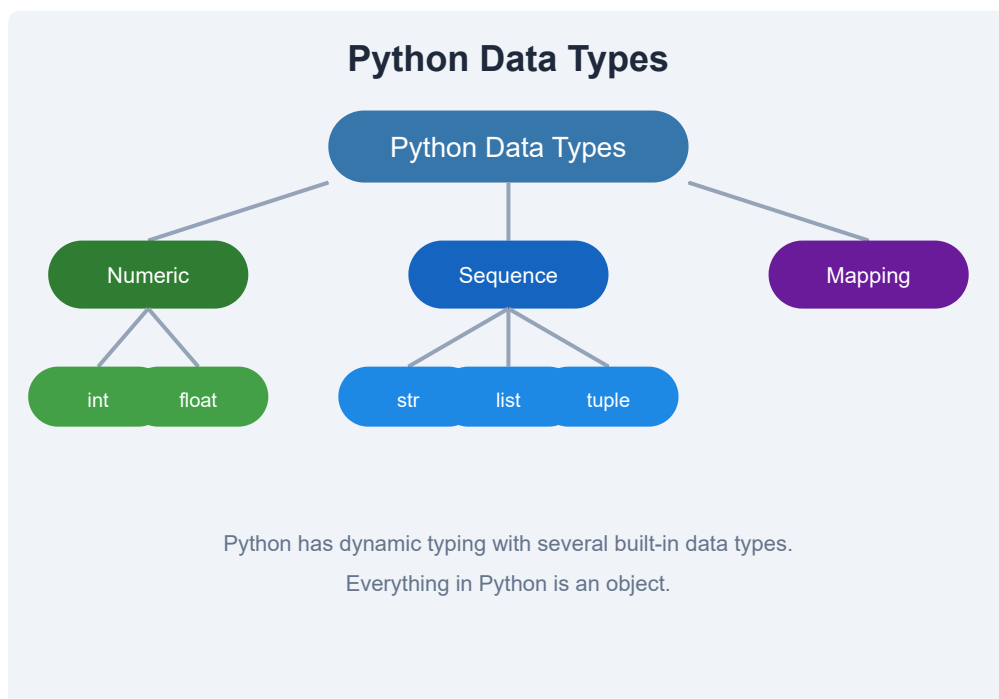
VS Code

Visual Studio Code with the Python extension (by Microsoft) is a popular, lightweight alternative. Install the Python extension from the marketplace and you will get linting, debugging, and IntelliSense.

Running Python Code

Whether you use an IDE or a simple text editor, the workflow is the same: write code, save the file, and run `python filename.py`.

```
# hello.py
name = input("What is your name? ")
print(f"Hello, {name}! Welcome to Python.")
```



Note: Python uses indentation (4 spaces by convention) to define blocks of code. Do not mix tabs and spaces.

Practice Exercise

Install Python if you haven't already. Create a new virtual environment called `learning`. Write a script that asks the user for their name, age, and favorite color, then prints a sentence combining all three pieces of information. Run the script both from the terminal and inside your chosen IDE.

Syntax & Control Flow

Basic Syntax

Python uses **indentation** to define code blocks instead of curly braces `{}`. The standard is 4 spaces per indentation level. Comments are written with `#`:

```
# This is a comment
x = 5 # inline comment
if x > 0:
    print("Positive") # indented block
```

Variables

Python is dynamically typed — you do not need to declare a variable's type. Its type is inferred at runtime.

```
name = "Alice"      # str
age = 30             # int
pi = 3.14159         # float
is_active = True     # bool
nothing = None       # NoneType
```

Multiple Assignment

```
a, b, c = 1, 2, 3
x = y = z = 0
```


The Private Convention

Variable names prefixed with an underscore (e.g. `_secret`) are conventionally considered "private" — a hint to other developers, not enforced by the language.

Data Types

Python's fundamental built-in types include:

- `int` — integer numbers (unbounded)
- `float` — floating-point numbers
- `str` — Unicode text
- `bool` — `True` or `False`
- `NoneType` — the `None` value (null)

Type Casting

```
int("42")      # 42
str(3.14)       # "3.14"
float("2.5")    # 2.5
bool(1)         # True
bool(0)         # False
```

Operators

Arithmetic

```
+ - * /      # add, subtract, multiply, divide
//           # floor division
%            # modulus (remainder)
**           # exponentiation
```

Comparison

```
==  !=  <  >  <=  >=
```

Logical

```
and  or  not
```

Assignment

```
=  +=  -=  *=  /=  //=  %=  **=
```

Bitwise

```
&  |  ^  ~  <<  >>
```

Conditionals: if / elif / else

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
```

match-case (Python 3.10+)

```
command = "quit"
match command:
    case "quit":
        print("Exiting...")
    case "help":
        print("Available commands: ...")
    case _:
        print(f"Unknown: {command}")
```

Loops

for Loops

```
# Range-based
for i in range(5):
    print(i)           # 0 1 2 3 4

# Iterate over a sequence
for char in "hello":
    print(char)

# With enumerate
for idx, val in enumerate(["a", "b", "c"]):
    print(idx, val)
```

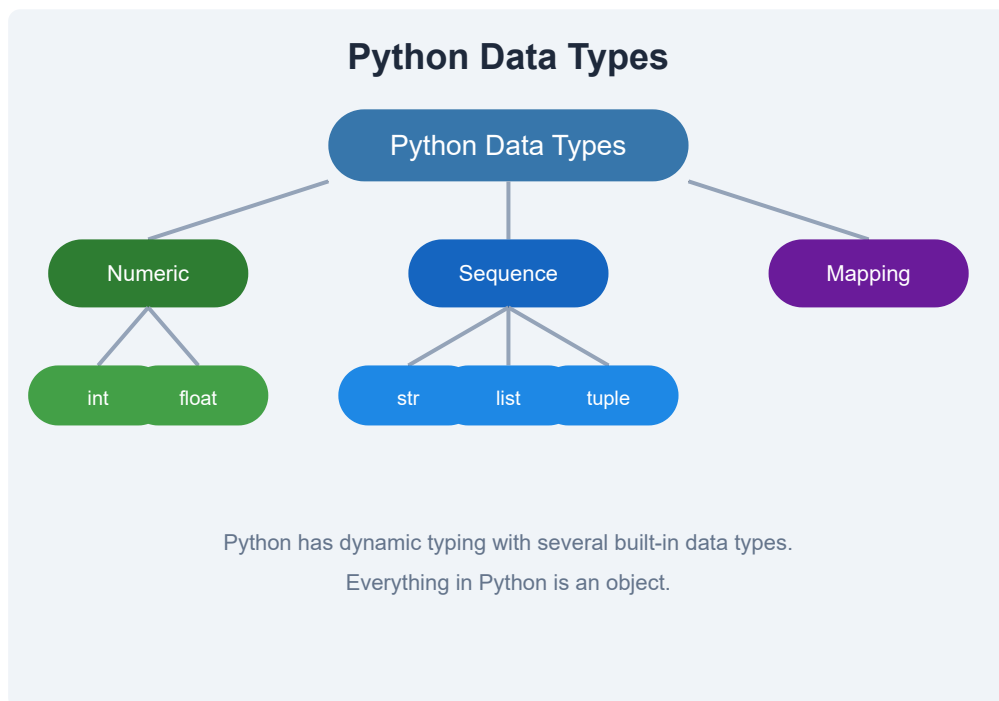
while Loops

```
count = 0
while count < 5:
    print(count)
    count += 1
```

break, continue, pass

```
for n in range(10):
    if n == 3:
        continue    # skip 3
    if n == 7:
        break       # stop at 7
    print(n)

# pass is a no-op placeholder
def unfinished_function():
    pass
```



Note: Python uses `elif` (not `else if` or `elsif`). Indentation errors are a common beginner mistake — always check your whitespace.

Practice Exercise

Write a program that asks the user for a number and prints whether it is positive, negative, or zero. Then extend it so the user can enter up to 5 numbers and the program prints the sum and average. Use a `for` loop and a `while` loop — try both versions.

Functions & Modules

Defining Functions

Functions are reusable blocks of code defined with `def`:

```
def greet(name):  
    """Return a greeting string."""  
    return f"Hello, {name}!"  
  
print(greet("Alice")) # Hello, Alice!
```

Parameters

Python offers flexible parameter options:

Positional & Keyword Arguments

```
def describe_pet(animal, name):  
    print(f"{name} is a {animal}")  
  
describe_pet("dog", "Rover")           # positional  
describe_pet(name="Fluffy", animal="cat") # keyword
```

Default Parameters

```
def power(base, exp=2):  
    return base ** exp  
  
print(power(3))      # 9 (3^2)  
print(power(3, 4))   # 81 (3^4)
```

Important: Default parameter values are evaluated once at function definition time, not each call. Never use mutable defaults like `[]` or `{}` unless you understand the implications.

***args (Variable Positional Arguments)**

```
def sum_all(*numbers):  
    return sum(numbers)  
  
print(sum_all(1, 2, 3, 4)) # 10
```

****kwargs (Variable Keyword Arguments)**

```
def print_info(**info):  
    for key, value in info.items():  
        print(f"{key}: {value}")  
  
print_info(name="Bob", age=25, city="NYC")
```

Return Values

Functions can return multiple values as a tuple:

```
def min_max(items):  
    return min(items), max(items)  
  
low, high = min_max([3, 1, 7, 2, 9])  
print(low, high) # 1 9
```

Scope (LEGB Rule)

Python resolves variable names in this order: **Local** → **Enclosing** → **Global** → **Built-in**.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)
    inner()

outer() # local
```

Lambda Functions

Anonymous one-line functions created with `lambda`:

```
square = lambda x: x ** 2
print(square(5)) # 25

pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
pairs.sort(key=lambda pair: pair[0])
print(pairs) # [(1, 'one'), (2, 'two'), (3, 'three')]
```

Function Annotations

```
def add(a: int, b: int) -> int:
    return a + b
```

Annotations are hints — they are not enforced at runtime but are useful for type checkers (e.g. mypy).

Modules and Imports

```
# Import a whole module
import math
print(math.sqrt(16))

# Import specific names
from random import randint, choice

# Import with alias
import datetime as dt

# Import all (discouraged)
from os import *
```

The `__name__` Variable

```
# mymodule.py
def main():
    print("Running directly")

if __name__ == "__main__":
    main()
```

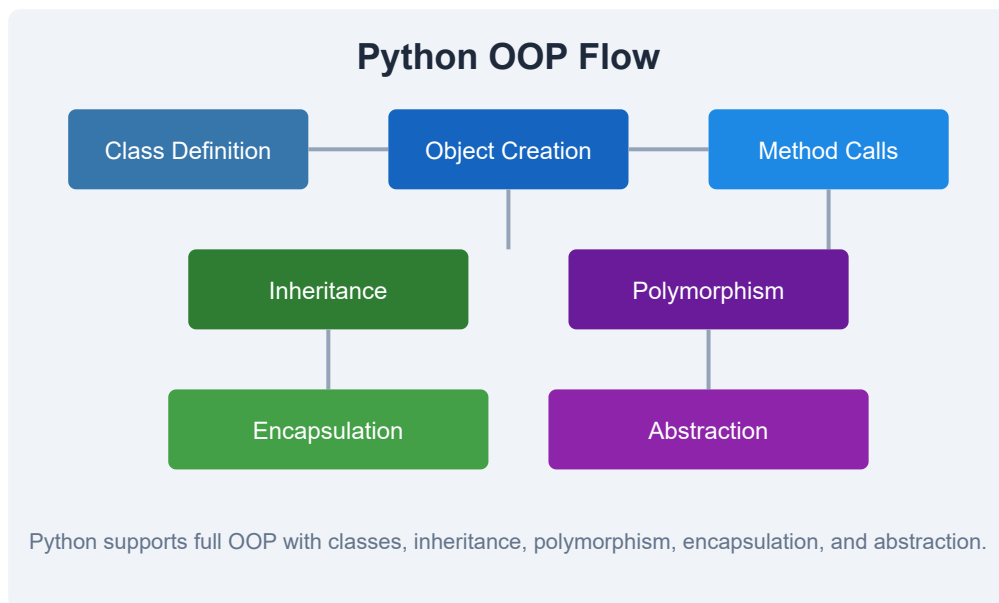
Packages

A package is a directory containing an `__init__.py` file (can be empty):

```
mypackage/
  __init__.py
  utils.py
  models.py
```

Useful Built-in Functions

```
len("hello")      # 5
range(10)         # 0..9
enumerate(["a","b"]) # (0,'a'), (1,'b')
zip([1,2], ["a","b"]) # (1,'a'), (2,'b')
sorted([3,1,2])   # [1,2,3]
map(str.upper, ["a","b"]) # ['A','B']
filter(lambda x: x>0, [-1,2,-3,4]) # [2,4]
```



Note: Use `if __name__ == "__main__"` to make a Python file both importable and runnable as a script.

Practice Exercise

Create a module `math_utils.py` with functions `add`, `subtract`, `multiply`, `divide`, and `power`. Add default parameter values where appropriate. Write a `main.py` that imports the module and demonstrates each function. Include a `if __name__` guard in both files.

Strings

String Basics

Strings are sequences of Unicode characters. You can write them with single quotes, double quotes, or triple quotes for multiline strings:

```
single = 'Hello'
double = "World"
triple = """This spans
multiple lines"""
also_triple = '''This works too'''
```

Slicing

Slicing extracts a portion using `[start:stop:step]`:

```
text = "Python"
print(text[0:2])    # Py
print(text[:3])     # Pyt
print(text[2:])     # thon
print(text[::-1])   # nohtyP (reverse)
print(text[::2])    # Pto (every other)
```

Immutability

Strings are immutable — you cannot change a character in place:

```
s = "hello"
# s[0] = "H" # TypeError!
s = "H" + s[1:] # "Hello" — creates a new string
```

Concatenation and Repetition

```
full = "Hello" + " " + "World" # "Hello World"
line = "-" * 20                 # "-----"
print("Hey" * 3)                # HeyHeyHey
```

String Formatting

f-strings (Python 3.6+)

```
name = "Alice"
age = 30
print(f"{name} is {age} years old")
print(f"{name!r}") # repr of name
print(f"{age:04d}") # "0030"
```

.format()

```
print("{} is {} years old".format("Bob", 25))
print("{1} is {0} years old".format(25, "Bob"))
```

% Operator (Legacy)

```
print("%s is %d years old" % ("Charlie", 35))
```

Escape Sequences

```
newline = "Line1\nLine2" # newline
tabbed = "Col1\tCol2"    # tab
quote = "She said \"Hi\"" # double quote
backslash = "C:\\Users\\name" # backslash
```

Use raw strings with `r"..."` to ignore escape sequences:

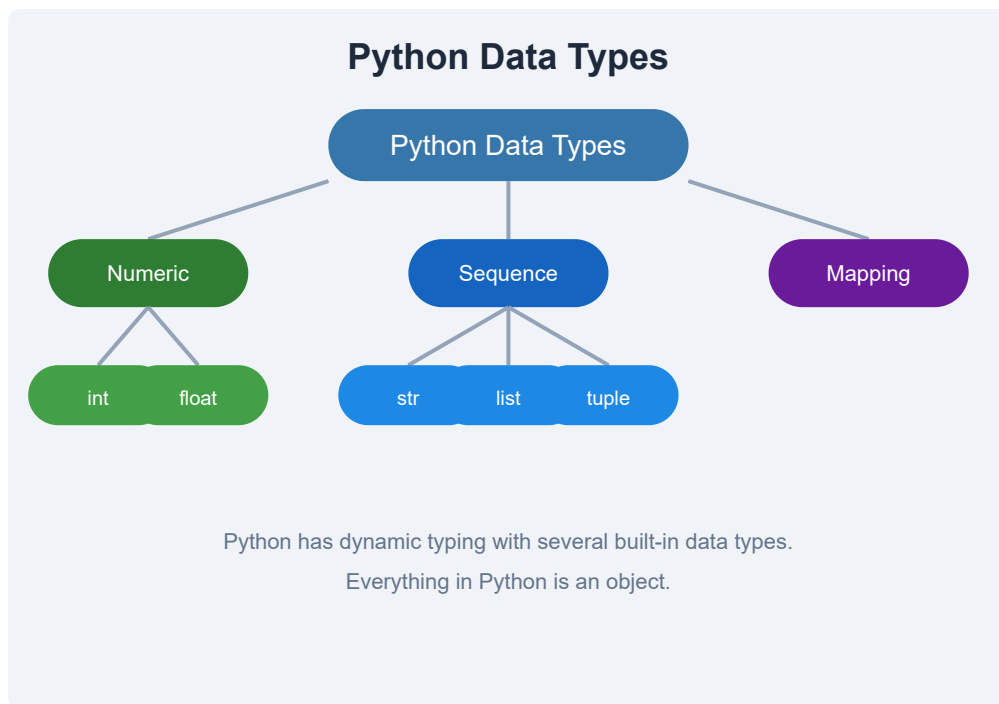
```
path = r"C:\Users\name" # no escaping needed
```

Common String Methods

```
text = " Hello, World! "  
  
text.upper()      # " HELLO, WORLD! "  
text.lower()      # " hello, world! "  
text.strip()      # "Hello, World!"  
text.split(",")   # [" Hello", " World! "]  
",".join(["a","b","c"]) # "a,b,c"  
text.replace("World", "Python") # " Hello, Python! "  
text.find("World") # 8  
text.count("l")    # 3  
text.startswith(" He") # True  
text.endswith("! ") # True
```

Checking String Content

```
"hello".isalpha() # True  
"123".isdigit()   # True  
"abc123".isalnum() # True  
" ".isspace()     # True  
"Hello".istitle() # True
```



Note: f-strings are the recommended way to format strings in modern Python. They are faster and more readable than `.format()` and the `%` operator.

Practice Exercise

Write a program that asks the user for a sentence. Print the number of characters, words, and vowels in the sentence. Then reverse the sentence word-by-word (e.g. "Hello world" → "world Hello"). Finally, replace all vowels with `*` and print the result.

Lists

List Basics

A list is an ordered, mutable collection of items enclosed in square brackets

`[]` :

```
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True]
empty = []
numbers = list(range(5)) # [0, 1, 2, 3, 4]
```

Indexing and Slicing

```
items = ["a", "b", "c", "d", "e"]
items[0]    # "a"
items[-1]   # "e"
items[1:3]   # ["b", "c"]
items[:3]    # ["a", "b", "c"]
items[::2]   # ["a", "c", "e"]
items[::-1]  # ["e", "d", "c", "b", "a"]
```

Mutability

Unlike strings, lists can be modified in place:

```
nums = [1, 2, 3]
nums[0] = 99
print(nums) # [99, 2, 3]
```

List Methods

```
lst = [1, 2, 3]

lst.append(4)          # [1, 2, 3, 4]
lst.extend([5, 6])     # [1, 2, 3, 4, 5, 6]
lst.insert(0, 0)       # [0, 1, 2, 3, 4, 5, 6]
lst.remove(3)          # removes first 3
popped = lst.pop()     # removes and returns last item
lst.pop(0)             # removes and returns index 0
lst.clear()            # []

lst = [3, 1, 4, 1, 5]
lst.index(4)           # 2
lst.count(1)           # 2
lst.sort()             # [1, 1, 3, 4, 5]
lst.sort(reverse=True) # [5, 4, 3, 1, 1]
lst.reverse()          # reverse in place
copy = lst.copy()      # shallow copy
```

List Comprehension

A concise way to create lists:

```
squares = [x ** 2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
pairs = [(a, b) for a in [1,2] for b in ["x","y"]]
# [(1, 'x'), (1, 'y'), (2, 'x'), (2, 'y')]
```


Looping Over Lists

```
colors = ["red", "green", "blue"]

for color in colors:
    print(color)

for i, color in enumerate(colors):
    print(i, color)

for a, b in zip([1,2], ["x","y"]):
    print(a, b)
```

Sorting

```
words = ["banana", "apple", "Cherry"]
words.sort(key=str.lower) # case-insensitive sort

students = [("Alice", 85), ("Bob", 92), ("Eve", 78)]
students.sort(key=lambda s: s[1], reverse=True)
# [("Bob", 92), ("Alice", 85), ("Eve", 78)]
```

Copying: Shallow vs Deep

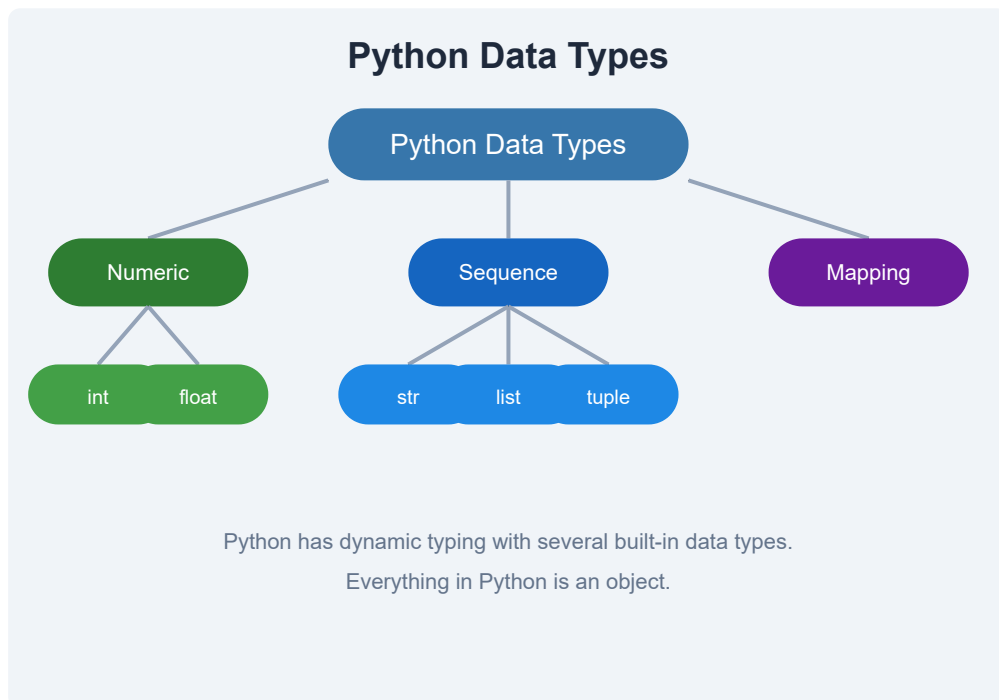
```
import copy

original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
shallow[0][0] = 99 # affects original too!

deep = copy.deepcopy(original)
deep[0][0] = 100 # original unchanged
```

Nested Lists

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(matrix[1][2]) # 6  
  
# Flatten a nested list  
flat = [item for row in matrix for item in row]  
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```



Note: List comprehensions are more Pythonic than `map()` and `filter()` in most cases. They are also faster than manual `for` loops with `append`.

Practice Exercise

Create a list of 10 random integers between 1 and 100. Write code that: (1) prints the list sorted, (2) prints the sum and average, (3) creates a new list containing only the even numbers using list comprehension, (4) finds the second largest number, and (5) reverses the list in place. Use at least 5 different list methods.

Tuples

Tuple Basics

A tuple is an ordered, **immutable** collection enclosed in parentheses `()`:

```
point = (3, 4)
rgb = (255, 128, 0)
single = (1,)      # trailing comma required
empty = ()
coordinates = 10, 20 # parentheses optional
```

Immutability

Once created, a tuple cannot be changed:

```
t = (1, 2, 3)
# t[0] = 99 # TypeError!
# t.append(4) # AttributeError!
# t.pop() # AttributeError!
```

However, if a tuple contains mutable objects, those objects can be modified:

```
t = ([1, 2], [3, 4])
t[0].append(99) # t is now ([1, 2, 99], [3, 4])
```

Accessing Elements

```
t = ("a", "b", "c", "d", "e")
t[0]      # "a"
t[-1]     # "e"
t[1:3]    # ("b", "c")
t[::-1]   # ("e", "d", "c", "b", "a")
```

Packing and Unpacking

```
# Packing
person = "Alice", 30, "Engineer"

# Unpacking
name, age, job = person
print(name)    # Alice
print(age)     # 30

# Extended unpacking (Python 3+)
first, *middle, last = (1, 2, 3, 4, 5)
print(first)   # 1
print(middle)  # [2, 3, 4]
print(last)    # 5
```

Nested Tuples

```
nested = ((1, 2), (3, 4), (5, 6))
print(nested[0][1]) # 2

# Flatten a nested tuple
flat = tuple(item for row in nested for item in row)
print(flat) # (1, 2, 3, 4, 5, 6)
```

namedtuple

The `collections.namedtuple` factory creates lightweight, immutable objects with named fields:

```
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p = Point(10, 20)
print(p.x)      # 10
print(p.y)      # 20
print(p[0])     # 10 (indexing still works)

Person = namedtuple("Person", "name age job")
alice = Person("Alice", 30, "Engineer")
print(alice.name) # Alice
```

Use `._replace()` to create a new instance with modified fields:

```
alice2 = alice._replace(age=31)
print(alice2) # Person(name='Alice', age=31, job='Engineer')
```

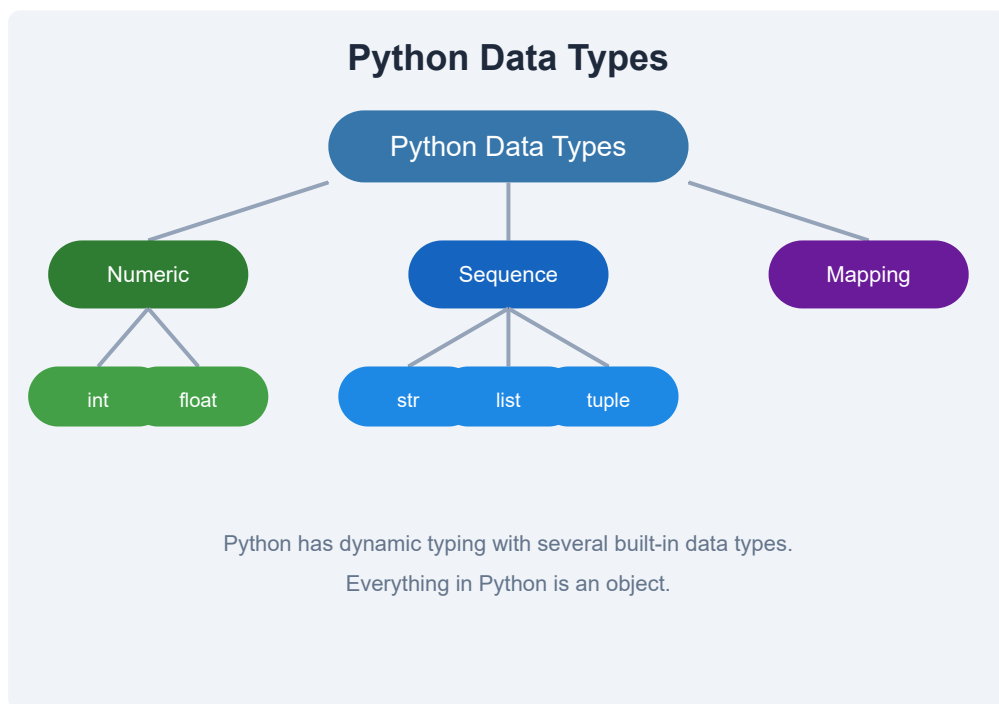
Tuple Methods

```
t = (1, 2, 3, 2, 4, 2)
t.count(2) # 3
t.index(3) # 2
# t.index(10) # ValueError: tuple.index(x): x not in tuple
```

When to Use Tuples vs Lists

Feature	Tuple	List
Mutable	No	Yes
Syntax	<code>()</code>	<code>[]</code>
Hashable	Yes (if contents are)	No
Use case	Fixed data, dict keys	Dynamic collections
Performance	Slightly faster	Slightly slower

Tip: Use tuples for data that should not change (e.g. coordinates, RGB values, database records). They are also commonly used to return multiple values from functions.



Note: A tuple with one element requires a trailing comma — `(1,)` not `(1)`. Without the comma, `(1)` is just the integer 1 in parentheses.

Practice Exercise

Create a `namedtuple` called `Student` with fields `name`, `grade`, and `subjects` (list of subject names). Create 3 students. Write a function that takes a tuple of students and returns the student with the highest grade. Print each student's details and the top student's info. Demonstrate packing, unpacking, and indexing.

Sets

Set Basics

A set is an unordered collection of **unique**, immutable elements enclosed in curly braces `{}`:

```
fruits = {"apple", "banana", "cherry"}
numbers = {1, 2, 3, 2, 1}  # {1, 2, 3} – duplicates removed
empty_set = set()         # not {} – that's a dict!
mixed = {1, "hello", (1, 2)}  # must be hashable elements
```

Adding and Removing Elements

```
s = {1, 2, 3}

s.add(4)          # {1, 2, 3, 4}
s.update([5, 6])  # {1, 2, 3, 4, 5, 6}
s.add(1)          # no change (already exists)

s.remove(3)       # {1, 2, 4, 5, 6}
# s.remove(99)    # KeyError!
s.discard(99)     # no error (safe remove)
popped = s.pop()  # removes and returns arbitrary element
s.clear()         # set()
```

Set Operations

Sets support mathematical set operations:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

a | b          # union: {1, 2, 3, 4, 5, 6}
a.union(b)     # same

a & b          # intersection: {3, 4}
a.intersection(b) # same

a - b          # difference: {1, 2}
a.difference(b) # same

a ^ b          # symmetric difference: {1, 2, 5, 6}
a.symmetric_difference(b) # same
```

Comparison Operators

```
{1, 2}.issubset({1, 2, 3}) # True
{1, 2} <= {1, 2, 3}       # True (subset)
{1, 2, 3}.issuperset({1, 2}) # True
{1, 2, 3} >= {1, 2}       # True (superset)
{1, 2}.isdisjoint({3, 4}) # True (no common elements)
```

Useful Set Methods

```
s = {3, 1, 4, 1, 5}
len(s)          # 4 (duplicates removed)
1 in s          # True
99 not in s     # True
copy = s.copy() # shallow copy
```

Set Comprehension

```
squares = {x ** 2 for x in range(10)}
evens = {x for x in range(20) if x % 2 == 0}
print(squares) # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

Frozenset

An immutable version of a set:

```
fs = frozenset([1, 2, 3, 2, 1])
# fs.add(4) # AttributeError!
# fs.remove(1) # AttributeError!

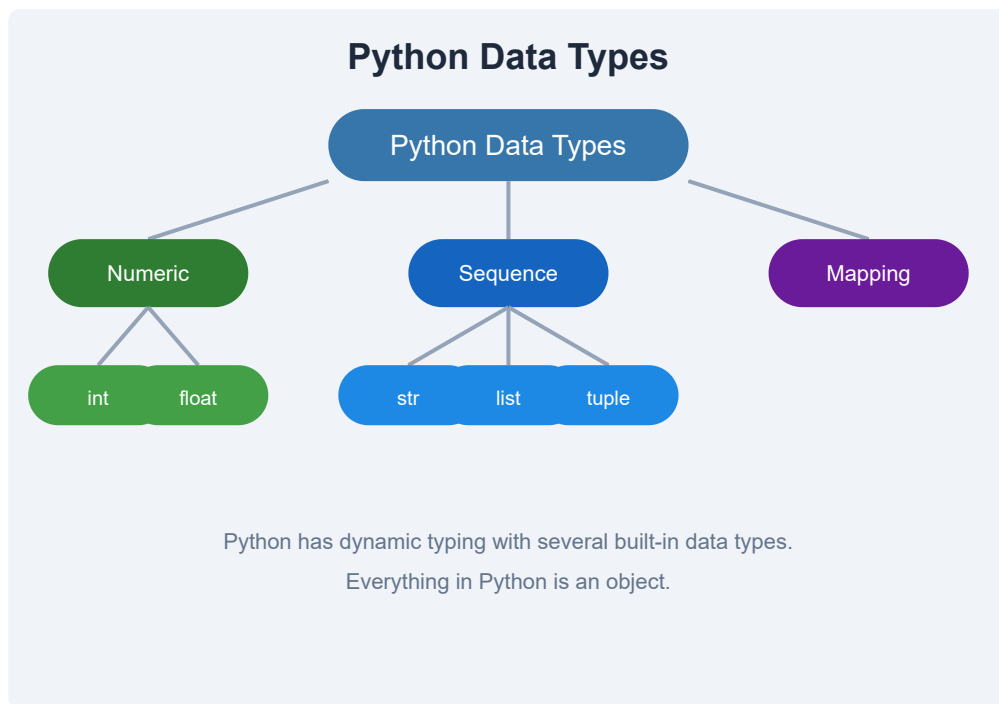
# Useful as dict keys or in other sets
sets_set = {frozenset([1, 2]), frozenset([3, 4])}
```

Common Use Cases

```
# Remove duplicates from a list
items = [1, 2, 3, 2, 1, 4, 3]
unique = list(set(items)) # [1, 2, 3, 4]

# Find common elements
friends_a = {"Alice", "Bob", "Charlie"}
friends_b = {"Bob", "Diana", "Eve"}
common = friends_a & friends_b # {"Bob"}

# Check if all items in one list are in another
required = {"name", "email", "age"}
provided = {"name", "email"}
missing = required - provided # {"age"}
```



Note: Sets are significantly faster than lists for membership tests (`in` / `not in`) because sets use hash tables internally. Use sets when order does not matter and uniqueness is required.

Practice Exercise

Create two lists of random numbers (10 numbers each, some overlapping). Convert them to sets and find: (1) all unique numbers across both lists, (2) numbers common to both, (3) numbers only in the first set, (4) numbers that appear in only one of the sets. Then write a function that takes two lists and returns the number of unique elements they share.

Dictionaries

Dictionary Basics

A dictionary stores key-value pairs, enclosed in curly braces `{}`:

```
student = {"name": "Alice", "age": 25, "grade": "A"}
empty = {}
pairs = dict([("a", 1), ("b", 2)])
kwargs = dict(name="Bob", age=30)
```

Accessing Values

```
d = {"name": "Alice", "age": 25}

d["name"]      # "Alice"
d.get("name")  # "Alice"
d.get("job")    # None (no error)
d.get("job", "N/A") # "N/A" (default)
# d["job"]      # KeyError!
```

Adding and Updating

```
d = {"a": 1}
d["b"] = 2      # add new key
d["a"] = 99     # update existing
d.update({"c": 3, "d": 4}) # merge
d.update(e=5, f=6) # keyword form
```

Removing Items

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}

d.pop("a")          # removes 'a' and returns 1
d.pop("x", None)     # returns None (safe)
d.popitem()         # removes & returns (key, value) pair
del d["b"]          # removes key
d.clear()           # empties dict
```

View Objects

```
d = {"name": "Alice", "age": 25, "grade": "A"}

d.keys()    # dict_keys(['name', 'age', 'grade'])
d.values()  # dict_values(['Alice', 25, 'A'])
d.items()   # dict_items([('name', 'Alice'), ('age', 25), ('grade', 'A')])
```

Looping Over Dictionaries

```
d = {"a": 1, "b": 2, "c": 3}

for key in d:
    print(key, d[key])

for key, value in d.items():
    print(key, value)

for value in d.values():
    print(value)
```

Checking Membership

```
d = {"name": "Alice", "age": 25}

"name" in d      # True (checks keys)
"Alice" in d     # False (not in keys)
"Alice" in d.values() # True
len(d)          # 2
```

Nested Dictionaries

```
users = {
    "alice": {"age": 25, "city": "NYC", "active": True},
    "bob": {"age": 30, "city": "LA", "active": False},
    "eve": {"age": 22, "city": "Chicago", "active": True},
}

print(users["alice"]["city"]) # NYC

# List active users
active = [name for name, info in users.items() if info["active"]]
print(active) # ['alice', 'eve']
```

Dictionary Comprehension

```
squares = {x: x ** 2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

even_squares = {x: x ** 2 for x in range(10) if x % 2 == 0}
# {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}

# Swap keys and values
original = {"a": 1, "b": 2}
reversed = {v: k for k, v in original.items()}
# {1: 'a', 2: 'b'}
```

defaultdict

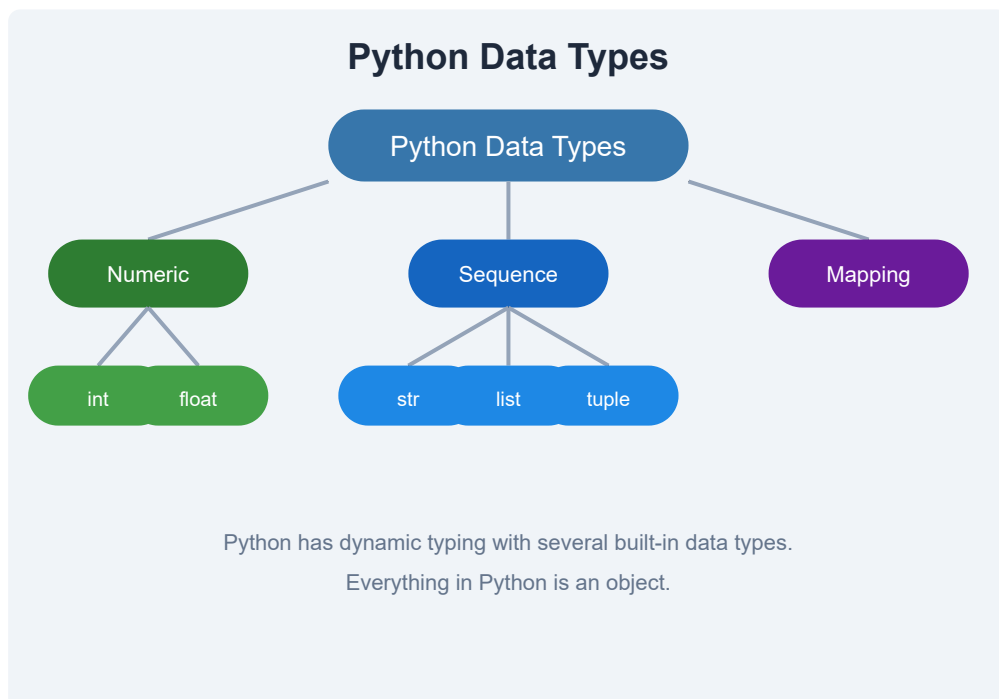
`collections.defaultdict` returns a default value for missing keys:

```
from collections import defaultdict

# Group words by first letter
words = ["apple", "banana", "avocado", "cherry", "blueberry"]
groups = defaultdict(list)
for word in words:
    groups[word[0]].append(word)

print(dict(groups))
# {'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

```
# Count occurrences
counts = defaultdict(int)
for char in "hello world":
    counts[char] += 1
print(dict(counts))
```



Note: Dictionary keys must be immutable types (strings, numbers, tuples of immutables). Lists and dictionaries cannot be used as keys. As of Python 3.7, dictionaries preserve insertion order.

Practice Exercise

Create a nested dictionary representing a phonebook with at least 5 contacts. Each contact has `name`, `phone`, `email`, and `groups` (list). Write code to: (1) find all contacts in a given group, (2) update a contact's phone number, (3) add a new contact, (4) print all contacts sorted by name, and (5) use a `defaultdict` to group contacts by their first letter.

Arrays

The array Module

Python's built-in `array` module provides space-efficient storage of basic C-like data types:

```
from array import array

# 'i' = signed int, 'f' = float, 'd' = double, 'u' = unicode char
numbers = array('i', [1, 2, 3, 4, 5])
print(numbers) # array('i', [1, 2, 3, 4, 5])
```

Type Codes

Code	C Type	Python Type	Min Size
'b'	signed char	int	1 byte
'B'	unsigned char	int	1 byte
'h'	signed short	int	2 bytes
'i'	signed int	int	2-4 bytes
'f'	float	float	4 bytes
'd'	double	float	8 bytes

Accessing Elements

Arrays support the same indexing and slicing syntax as lists:

```
arr = array('i', [10, 20, 30, 40, 50])
arr[0]      # 10
arr[-1]     # 50
arr[1:3]    # array('i', [20, 30])
arr[:2]     # array('i', [10, 30, 50])
arr[0] = 99 # mutable: [99, 20, 30, 40, 50]
```

Looping

```
arr = array('f', [1.5, 2.5, 3.5])

for val in arr:
    print(val)

for i, val in enumerate(arr):
    print(i, val)

squared = [x ** 2 for x in arr] # list comprehension
```

Adding Elements

```
arr = array('i', [1, 2, 3])

arr.append(4)      # [1, 2, 3, 4]
arr.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
arr.insert(0, 0)   # [0, 1, 2, 3, 4, 5, 6]
```

Removing Elements

```
arr = array('i', [1, 2, 3, 4, 5, 3])

arr.pop()          # removes & returns 5
arr.pop(0)         # removes & returns 1
arr.remove(3)      # removes first occurrence of 3
# arr.remove(99)   # ValueError
arr.clear()        # empties array
```

Sorting and Copying

```
arr = array('i', [3, 1, 4, 1, 5, 9, 2])

# Arrays don't have a .sort() method
sorted_arr = array('i', sorted(arr))
print(sorted_arr) # array('i', [1, 1, 2, 3, 4, 5, 9])

# Reverse in place
arr.reverse()
print(arr)

# Copy
shallow = arr[:]      # slice copy
from copy import copy
shallow2 = copy(arr)  # shallow copy
deep = arr.__copy__() # also shallow
```

Arrays vs Lists

Feature	array	list
Type constraint	Single type	Any type
Memory	Compact (C-level)	Larger (PyObject*)
Flexibility	Limited methods	Rich methods
Comprehensions	Via list then convert	Built-in
Performance	Faster for homogeneous numeric data	More overhead per element

When to use arrays: Use `array` when you need to store a large number of numeric values of the same type and memory efficiency matters. For most general-purpose work, lists are more flexible and idiomatic.

Practical Example

```
from array import array

# Store sensor readings efficiently
readings = array('d', [22.5, 23.1, 21.8, 24.0, 22.9])

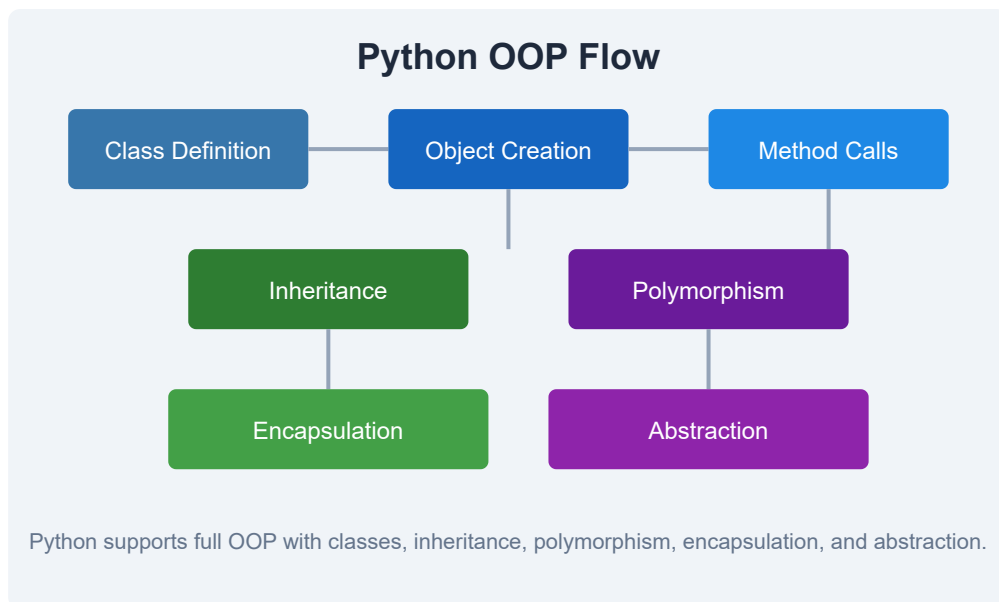
mean = sum(readings) / len(readings)
variance = sum((x - mean) ** 2 for x in readings) / len(readings)
std_dev = variance ** 0.5

print(f"Mean: {mean:.2f}, Std Dev: {std_dev:.2f}")

# Write to binary file
with open("readings.bin", "wb") as f:
    readings.tofile(f)

# Read back
restored = array('d')
with open("readings.bin", "rb") as f:
    restored.fromfile(f, 5)

print(restored) # array('d', [22.5, 23.1, 21.8, 24.0, 22.9])
```



Note: For heavy numerical work, consider NumPy (`numpy.ndarray`), which is far more powerful than the built-in `array` module. The `array` module is best for simple typed storage without external dependencies.

Practice Exercise

Create an array of 20 random integers between 0 and 100. Write code to: (1) find the minimum, maximum, and average values, (2) create a new array containing only values above the average, (3) remove all odd values from the original array, (4) sort the original array in ascending order using `sorted()` , and (5) write the sorted array to a binary file and read it back. Compare the memory usage with an equivalent list using `sys.getsizeof()` .

File Handling

Opening Files with `open()`

The built-in `open()` function opens a file and returns a file object. The mode determines the operation:

```
f = open("file.txt", "r")    # read (default)
f = open("file.txt", "w")    # write (overwrites!)
f = open("file.txt", "a")    # append
f = open("file.txt", "x")    # exclusive create (fails if exists)
f = open("file.txt", "rb")   # read binary
f = open("file.txt", "wb")   # write binary
```

Reading Files

```
# Read entire file as a string
with open("example.txt", "r") as f:
    content = f.read()

# Read one line at a time
with open("example.txt", "r") as f:
    line = f.readline()
    while line:
        print(line.strip())
        line = f.readline()

# Read all lines into a list
with open("example.txt", "r") as f:
    lines = f.readlines() # includes \n characters

# Iterate directly (memory-efficient)
with open("example.txt", "r") as f:
    for line in f:
        print(line.strip())
```

Writing Files

```
# write() writes a single string
with open("output.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("Second line\n")

# writelines() writes an iterable of strings
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)
```

Tip: Use `"w"` mode carefully — it overwrites the file without warning. For safety, check if the file exists first with `os.path.exists()` or use `"x"` mode.

The with Context Manager

The `with` statement automatically closes the file, even if an exception occurs:

```
# Bad: manual close
f = open("file.txt", "r")
data = f.read()
f.close()

# Good: automatic close
with open("file.txt", "r") as f:
    data = f.read()
# File is closed automatically
```

File Position: seek() and tell()

```
with open("example.txt", "r") as f:
    f.read(10)          # read first 10 chars
    pos = f.tell()      # current position (10)
    f.seek(0)           # go back to beginning
    f.seek(5)           # go to byte 5
    f.seek(0, 2)        # seek to end (2 = whence end)
```

The os Module

```
import os

os.rename("old.txt", "new.txt")  # rename file
os.remove("file.txt")           # delete file
os.mkdir("newdir")              # create directory
os.rmdir("emptydir")            # remove empty directory
os.listdir(".")                 # list directory contents
os.path.exists("file.txt")      # check existence
os.path.isfile("file.txt")      # is it a file?
os.path.isdir("mydir")          # is it a directory?
os.path.join("dir", "file.txt") # platform-aware path join
```

The pathlib Module (Modern)

`pathlib` provides an object-oriented approach to filesystem paths:

```
from pathlib import Path

p = Path("data/subdir")
p.mkdir(parents=True, exist_ok=True) # create dirs recursively

file = p / "notes.txt"           # path join with /
file.write_text("Hello, Path!")  # write text
content = file.read_text()       # read text
file.exists()                    # True
file.is_file()                   # True
file.suffix                      # '.txt'
file.stem                       # 'notes'
file.name                       # 'notes.txt'

# Glob pattern matching
txt_files = Path(".").glob("*.txt")
py_files = Path("src").rglob("*.py") # recursive

# Delete file
file.unlink()
```

Practical Example

```
from pathlib import Path

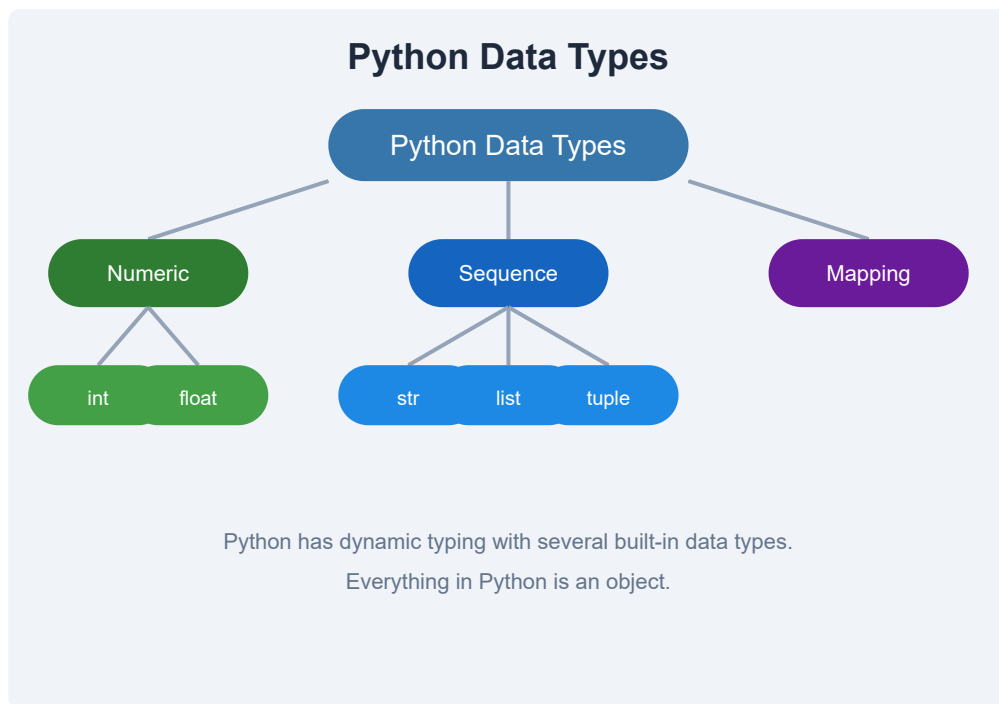
# Create a log file and append entries
log = Path("app.log")
entries = [
    "[INFO] Starting application",
    "[INFO] Connected to database",
    "[WARN] Memory usage above 80%",
    "[ERROR] Connection timeout",
]

with log.open("a") as f:
    for entry in entries:
        f.write(entry + "\n")

# Read and filter errors
errors = [line for line in log.read_text().splitlines()
          if "ERROR" in line]
print(f"Found {len(errors)} errors:")
for err in errors:
    print(f"  {err}")

# Backup the log
backup = log.parent / "backups" / f"app_{Path('log').stem}.bak"
backup.parent.mkdir(exist_ok=True)
backup.write_text(log.read_text())

# Clean up old backup files
for old in backup.parent.glob("*.bak"):
    if old.stat().st_mtime < (time.time() - 86400 * 7):
        old.unlink()
```

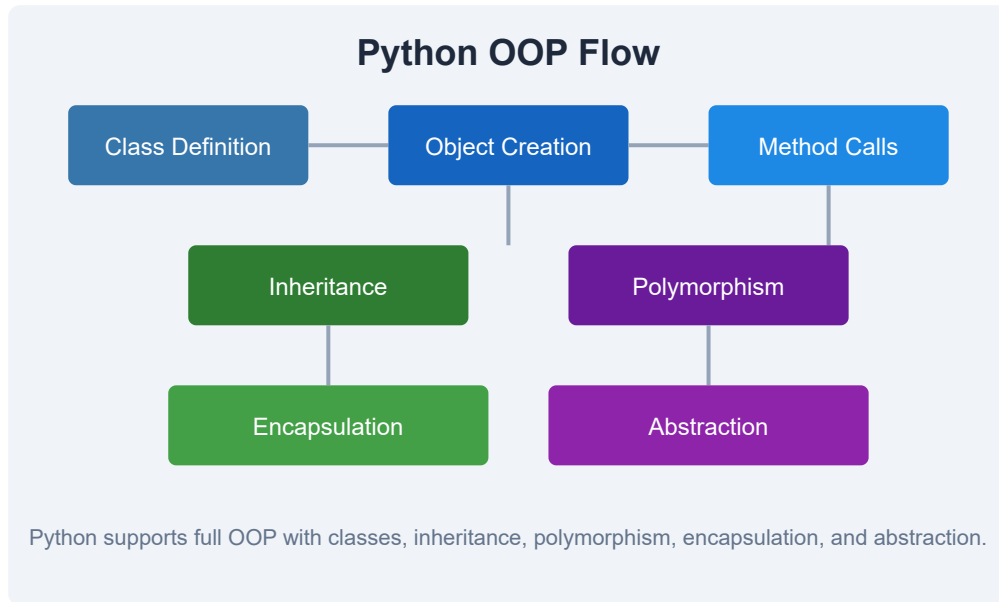


Note: Always prefer `with` statements for file operations. They guarantee proper resource cleanup. For new projects, `pathlib` is the recommended approach over `os.path` for path manipulation.

Practice Exercise

Write a program that: (1) creates a directory called `notes` if it doesn't exist, (2) creates 5 text files in it, each containing a numbered note, (3) reads all the files and concatenates their content into a single `summary.txt`, (4) prints the file names and sizes of all `.txt` files in the directory, and (5) then deletes the note files (keeping the directory). Use `pathlib` for all path operations.

Object-Oriented Programming (OOP)



Classes and Objects

A class is a blueprint for creating objects. In Python, everything is an object, and classes define the structure and behavior of those objects.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says woof!"

my_dog = Dog("Rex", 3)
print(my_dog.bark())
```

The `__init__` Method and `self`

The `__init__` method is the constructor, called automatically when a new instance is created. The `self` parameter refers to the current instance.

Instance vs Class Attributes

```
class Car:
    wheels = 4 # class attribute

    def __init__(self, make, model):
        self.make = make # instance attribute
        self.model = model
```

Instance, Class, and Static Methods

```
class MathUtils:
    @classmethod
    def from_string(cls, text):
        return cls()

    @staticmethod
    def add(a, b):
        return a + b
```

Property Decorators

The `@property` decorator allows you to define methods that can be accessed like attributes, with getter, setter, and deleter.

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def fahrenheit(self):
        return self._celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5 / 9
```

`__str__` and `__repr__`

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}')"

    def __str__(self):
        return f"{self.title} by {self.author}"
```

Inheritance and MRO

Inheritance allows a class to derive attributes and methods from a parent class. Python uses the C3 linearization algorithm for Method Resolution Order (MRO).

```
class Animal:
    def speak(self):
        return "Some sound"

class Cat(Animal):
    def speak(self):
        return "Meow"

print(Cat().speak()) # Meow
print(Cat.__mro__)   # Method resolution order
```

super()

```
class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)
```

Polymorphism and Method Overriding

Polymorphism lets different classes define methods with the same name. Method overriding allows a subclass to provide a specific implementation of a method already defined in its parent class.

```
for animal in [Dog(), Cat()]:
    print(animal.speak())
```

Encapsulation

Python uses naming conventions for access control: `_protected` (single underscore) and `__private` (double underscore triggers name mangling).


```
class BankAccount:
    def __init__(self):
        self.balance = 0          # public
        self._fee = 0.01         # protected
        self.__pin = "1234"      # private
```

Abstract Classes (ABC)

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Dataclasses

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

    def distance_from_origin(self):
        return (self.x**2 + self.y**2) ** 0.5
```

Enum

```
from enum import Enum

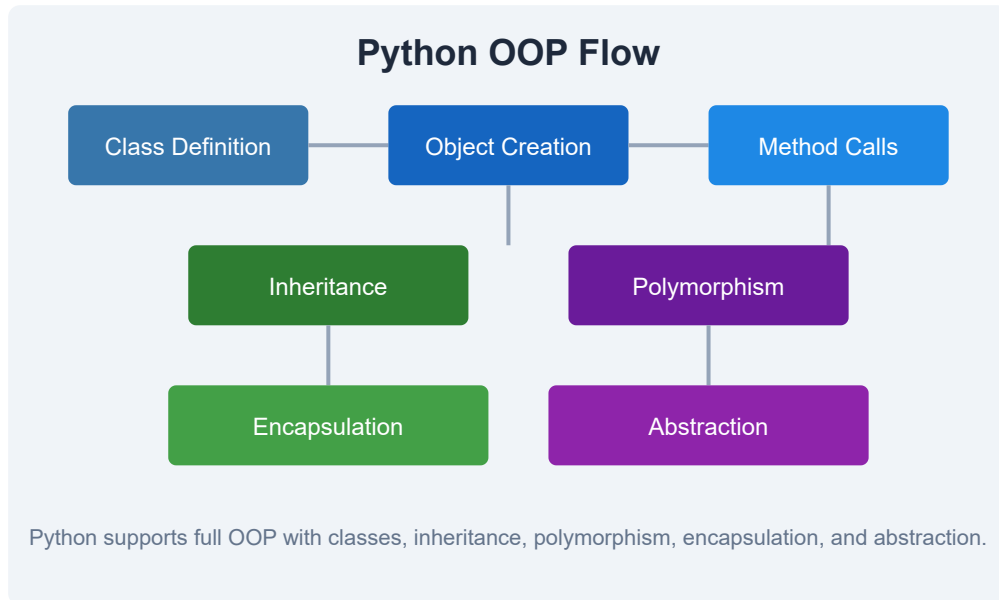
class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3
```

Note: Python supports multiple inheritance. Use it carefully to avoid the diamond problem.

Best Practice: Favor composition over inheritance when possible. Use ABCs to define interfaces.

Exercise: Create a `Library` class that stores `Book` objects. Implement methods to add, remove, search by title, and list all books. Use a dataclass for `Book` with fields: title, author, year, isbn.

Errors & Exceptions



Try, Except, Else, Finally

Python uses `try/except` blocks to handle runtime errors gracefully. The `else` clause runs when no exception occurs, and `finally` always executes.

```
try:
    result = 10 / int(user_input)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Invalid number!")
else:
    print(f"Result: {result}")
finally:
    print("Execution complete.")
```

Catching Specific Exceptions

Always catch specific exceptions rather than a bare `except:`. Common built-in exceptions include `ValueError`, `TypeError`, `FileNotFoundError`, `IndexError`, and `KeyError`.

```
try:
    with open("data.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found. Creating a new one.")
except PermissionError:
    print("Permission denied.")
```

Raising Exceptions with raise

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount
```

Assertions

The `assert` statement is used for debugging. It raises `AssertionError` if the condition is `False`.

```
def divide(a, b):
    assert b != 0, "Denominator cannot be zero"
    return a / b
```

Note: Assertions can be disabled with the `-O` flag. Do not use them for validation in production.

Custom Exceptions

```
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Need {amount}, have {balance}")

raise InsufficientBalanceError(100, 200)
```

Logging with the logging Module

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.debug("Detailed debug info")
logging.info("Operation completed successfully")
logging.warning("Low disk space")
logging.error("Failed to connect")
logging.critical("System is shutting down")
```

The warnings Module

```
import warnings

warnings.filterwarnings("ignore")
warnings.warn("This feature is deprecated", DeprecationWarning)
```

Debugging with pdb

Python's built-in debugger allows you to step through code interactively.

```
import pdb

def buggy_function(x):
    pdb.set_trace() # Execution pauses here
    result = x / 0
    return result
```

In Python 3.7+, you can use the built-in `breakpoint()` function instead of `pdb.set_trace()`.

```
def calculate(values):
    breakpoint() # Opens the debugger
    return sum(values) / len(values)
```

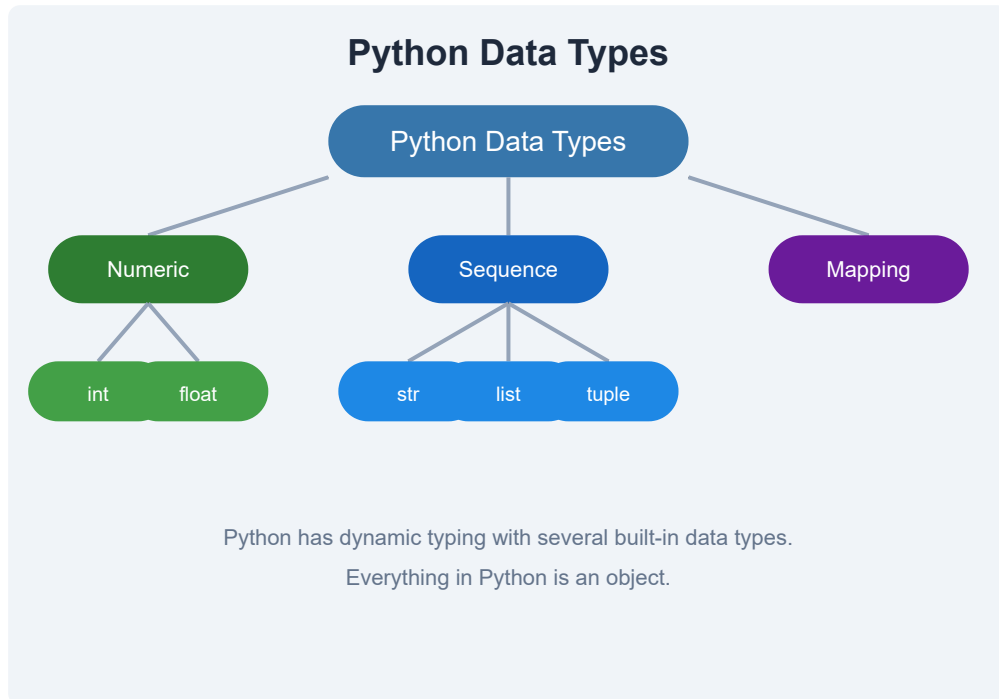
Tip: Common pdb commands: `n` (next line), `s` (step into), `c` (continue), `p var` (print variable), `l` (list source), `q` (quit).

Exception Chaining

```
try:
    process_file("data.csv")
except FileNotFoundError as e:
    raise RuntimeError("Cannot start processing") from e
```

Exercise: Write a function `safe_divide(a, b)` that handles `ZeroDivisionError`, `TypeError`, and logs errors using the logging module. Then write a custom exception `ValidationError` and use it in a function that validates user input (age between 0-120, name non-empty).

Multithreading



The Thread Class

The `threading` module provides a high-level interface for threading. A thread is a separate flow of execution within a process.

```
import threading
import time

def worker(name, delay):
    for i in range(3):
        print(f"{name}: iteration {i}")
        time.sleep(delay)

t1 = threading.Thread(target=worker, args=("Thread-A", 0.5))
t2 = threading.Thread(target=worker, args=("Thread-B", 0.3))

t1.start()
t2.start()

t1.join()
t2.join()
print("Both threads completed.")
```

Daemon Threads

Daemon threads run in the background and automatically exit when the main program ends. They are useful for monitoring or housekeeping tasks.

```
daemon = threading.Thread(target=monitor, daemon=True)
daemon.start()
```

Thread Synchronization with Lock

Locks prevent race conditions by ensuring only one thread accesses a shared resource at a time.


```

lock = threading.Lock()
counter = 0

def increment():
    global counter
    for _ in range(100000):
        lock.acquire()
        counter += 1
        lock.release()

# Using context manager (preferred)
def decrement():
    global counter
    for _ in range(100000):
        with lock:
            counter -= 1

```

RLock (Reentrant Lock)

An `RLock` can be acquired multiple times by the same thread without blocking. Useful when a function calls another function that needs the same lock.

```

rlock = threading.RLock()

def outer():
    with rlock:
        inner()

def inner():
    with rlock:
        print("Inside inner function")

```

Semaphore

A `Semaphore` limits access to a resource to a fixed number of threads at a time.

```
pool = threading.Semaphore(3)

def limited_task(n):
    with pool:
        print(f"Task {n} running")
        time.sleep(1)
```

ThreadPoolExecutor (concurrent.futures)

The `ThreadPoolExecutor` provides a simpler, higher-level interface for managing a pool of threads.

```
from concurrent.futures import ThreadPoolExecutor

def fetch_url(url):
    # Simulate fetching a URL
    time.sleep(1)
    return f"Data from {url}"

urls = ["http://example.com", "http://test.org", "http://demo.net"]

with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(fetch_url, urls)

for result in results:
    print(result)
```

The Global Interpreter Lock (GIL)

The GIL is a mutex in CPython that prevents multiple threads from executing Python bytecode simultaneously. This means CPU-bound tasks do not benefit from threading — use `multiprocessing` instead. Threading is still useful for I/O-bound tasks.

Note: For CPU-bound operations, use the `multiprocessing` module which creates separate processes, each with its own GIL.

Threading vs Multiprocessing

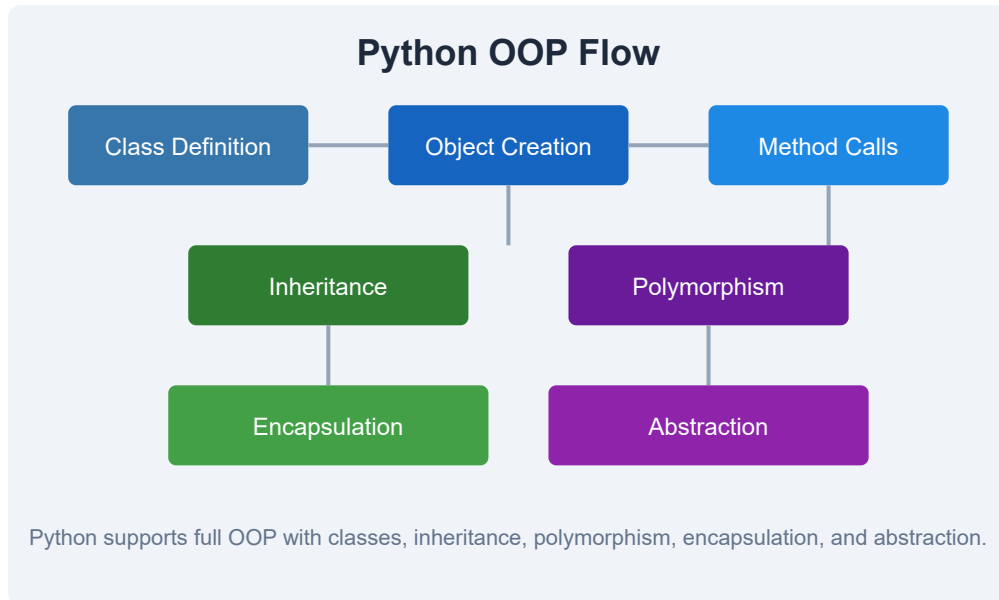
```
# Threading - good for I/O-bound tasks
import threading
threading.Thread(target=io_task).start()

# Multiprocessing - good for CPU-bound tasks
import multiprocessing
multiprocessing.Process(target=cpu_task).start()
```

Best Practice: Use `ThreadPoolExecutor` for I/O-bound tasks and `ProcessPoolExecutor` for CPU-bound tasks to keep your code clean and maintainable.

Exercise: Write a program that downloads 10 URLs concurrently using `ThreadPoolExecutor`. Measure the total time and compare it with sequential execution. Then implement a thread-safe counter using a `Lock` and verify it works correctly with 100 threads incrementing it 1000 times each.

Networking



Socket Programming

The `socket` module provides low-level networking interfaces. Sockets are endpoints for sending and receiving data across a network.

TCP Server

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 8080))
server.listen(5)
print("Server listening on port 8080...")

while True:
    client, addr = server.accept()
    print(f"Connected by {addr}")
    client.send(b"Hello from server!")
    client.close()
```

TCP Client

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 8080))
data = client.recv(1024)
print(f"Received: {data.decode()}")
client.close()
```

HTTP Requests with urllib.request

```
from urllib.request import urlopen

with urlopen("https://api.github.com") as response:
    data = response.read()
    print(response.status)
    print(data[:200])
```

The requests Library

The `requests` library simplifies HTTP requests significantly compared to the built-in `urllib`.

```
import requests

# GET request
response = requests.get("https://api.github.com/users/python")
print(response.status_code)
print(response.json()["login"])

# POST request with JSON
payload = {"title": "Hello", "body": "World"}
r = requests.post("https://jsonplaceholder.typicode.com/posts", json=payload)
print(r.status_code, r.json())
```

Handling HTTP Responses

```
response = requests.get("https://httpbin.org/get", params={"key": "value"})
print(response.text)           # Raw text
print(response.json())         # Parsed JSON
print(response.headers)        # Response headers
print(response.elapsed)        # Request duration
```

Note: Always handle potential network errors such as `requests.ConnectionError`, `Timeout`, and `HTTPError`.

Error Handling in Network Requests

```
import requests
from requests.exceptions import RequestException

try:
    r = requests.get("https://example.com", timeout=5)
    r.raise_for_status()
except RequestException as e:
    print(f"Request failed: {e}")
```

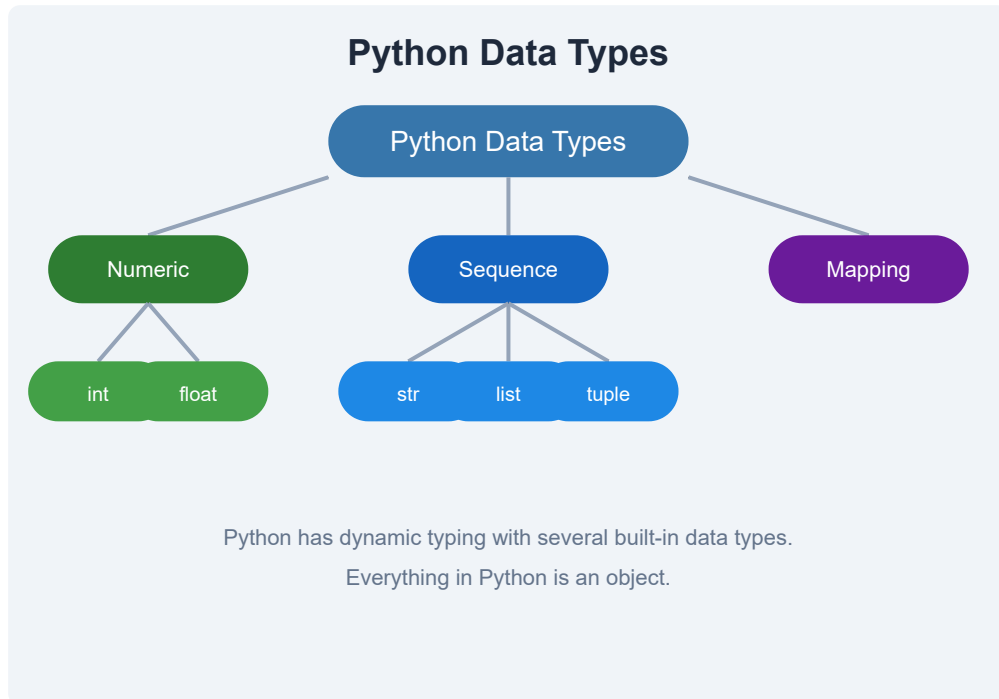
Session Objects for Multiple Requests

```
with requests.Session() as session:
    session.headers.update({"Authorization": "Bearer token123"})
    r1 = session.get("https://api.example.com/user")
    r2 = session.get("https://api.example.com/posts")
```

Tip: Use `timeout` parameter in all requests to prevent your program from hanging indefinitely.

Exercise: Build a simple chat application using TCP sockets where a server can handle multiple clients (use threading). The server should broadcast messages from one client to all connected clients. Also write a script that fetches data from a public API (e.g., JSONPlaceholder), processes the JSON response, and handles connection errors gracefully.

Libraries



NumPy

NumPy is the fundamental package for scientific computing in Python. It provides the `ndarray` object for efficient array operations.


```

import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr.shape)      # (5,)
print(arr.reshape(5, 1))

zeros = np.zeros((3, 4))
ones = np.ones((2, 3))
range_arr = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]

matrix = np.array([[1, 2], [3, 4]])
print(matrix + 10)      # Broadcasting
print(matrix @ matrix.T) # Matrix multiplication

```

Pandas

Pandas provides `Series` and `DataFrame` for data manipulation and analysis.

```

import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "Salary": [50000, 60000, 70000]
}

df = pd.DataFrame(data)
print(df.head())
print(df.describe())

grouped = df.groupby("Age").mean()
merged = pd.merge(df, other_df, on="id")

df.to_csv("output.csv", index=False)
read_df = pd.read_csv("data.csv")

```

SciPy

SciPy builds on NumPy and provides modules for optimization, statistics, integration, and signal processing.

```
from scipy.optimize import minimize
from scipy import stats
from scipy.integrate import quad

# Optimization
result = minimize(lambda x: x**2 + 10, x0=2)
print(result.x)

# Statistics
data = [1, 2, 3, 4, 5]
mean = stats.tmean(data)
z_scores = stats.zscore(data)

# Integration
area, error = quad(lambda x: x**2, 0, 1)
print(f"Area under x^2 from 0 to 1: {area}")
```

Matplotlib

Matplotlib is the standard plotting library in Python.

```

import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker="o", linestyle="--", color="b")
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()

# Bar chart and histogram
categories = ["A", "B", "C"]
values = [3, 7, 5]
plt.bar(categories, values)

data = [1, 2, 2, 3, 3, 3, 4, 4, 5]
plt.hist(data, bins=5)

# Subplots
fig, axes = plt.subplots(2, 2)
axes[0, 0].plot(x, y)
axes[0, 1].bar(categories, values)
plt.tight_layout()

```

Django Overview

Django is a high-level web framework following the MTV (Model-Template-View) pattern. It includes an ORM, admin interface, URL routing, and templating.

```

# models.py
from django.db import models

class BlogPost(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    pub_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

# views.py
from django.shortcuts import render
from .models import BlogPost

def home(request):
    posts = BlogPost.objects.all()
    return render(request, "home.html", {"posts": posts})

# urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]

```

OpenCV Basics

```

import cv2

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)

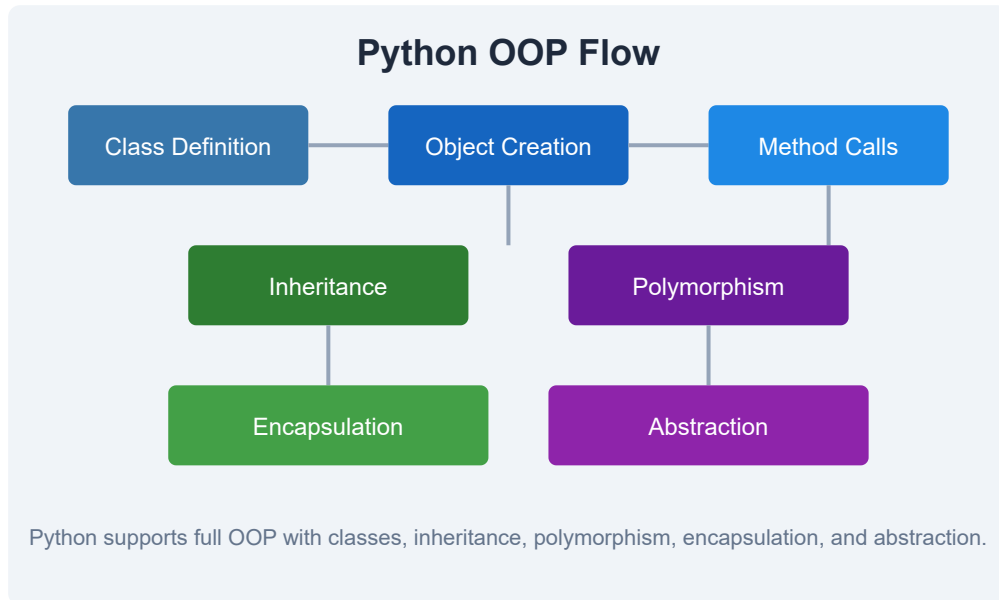
cv2.imshow("Original", img)
cv2.imshow("Edges", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Note: Install these libraries with `pip install numpy pandas scipy matplotlib django opencv-python` .

Exercise: Load a CSV dataset using Pandas, compute summary statistics, group data by a categorical column, and plot a histogram and a bar chart using Matplotlib. Then use SciPy to perform a t-test on two groups from the data.

Miscellaneous Topics



datetime Module

```
from datetime import datetime, date, time, timedelta

now = datetime.now()
today = date.today()
current_time = time(14, 30, 0)

delta = timedelta(days=7)
next_week = now + delta

formatted = now.strftime("%Y-%m-%d %H:%M:%S")
parsed = datetime.strptime("2025-01-15", "%Y-%m-%d")
```

math Module

```
import math

print(math.pi, math.e)
print(math.sqrt(16))      # 4.0
print(math.factorial(5))  # 120
print(math.gcd(12, 18))   # 6
print(math.ceil(4.2))     # 5
print(math.floor(4.8))    # 4
```

Iterators and Generators

An iterator implements `__iter__` and `__next__`. A generator uses `yield` to produce a sequence lazily.

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.limit:
            raise StopIteration
        self.count += 1
        return self.count

# Generator
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

# Generator expression
squares = (x**2 for x in range(10))
```

Closures and Decorators

```
# Closure
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

double = make_multiplier(2)
print(double(5)) # 10

# Decorator
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@timer
def slow_function():
    import time
    time.sleep(1)
    return "Done"
```


Recursion

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def binary_search(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)
```

Regular Expressions (re)

```
import re

text = "Contact us at support@example.com or sales@test.org"

emails = re.findall(r"\b[\w.]+@[ \w.]+\.\w+\b", text)
print(emails)

if re.search(r"\d{3}-\d{4}", "Call 555-1234"):
    print("Phone number found")

cleaned = re.sub(r"^[a-zA-Z0-9]", "-", "Hello World! 2025")
print(cleaned)
```

pip and requirements.txt

```
pip install requests numpy pandas
pip freeze > requirements.txt
pip install -r requirements.txt
pip list --outdated
```

SQLite

```
import sqlite3

conn = sqlite3.connect("database.db")
cursor = conn.cursor()

cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT)")
cursor.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))
conn.commit()

cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()
for row in rows:
    print(row)

conn.close()
```

JSON

```
import json

data = {"name": "Alice", "age": 30, "skills": ["Python", "SQL"]}

json_str = json.dumps(data, indent=2)
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

with open("data.json") as f:
    loaded = json.load(f)
```

Tkinter Basics

```
import tkinter as tk

root = tk.Tk()
root.title("My App")
root.geometry("300x200")

label = tk.Label(root, text="Hello, Tkinter!")
label.pack()

button = tk.Button(root, text="Click Me", command=root.quit)
button.pack()

root.mainloop()
```

argparse

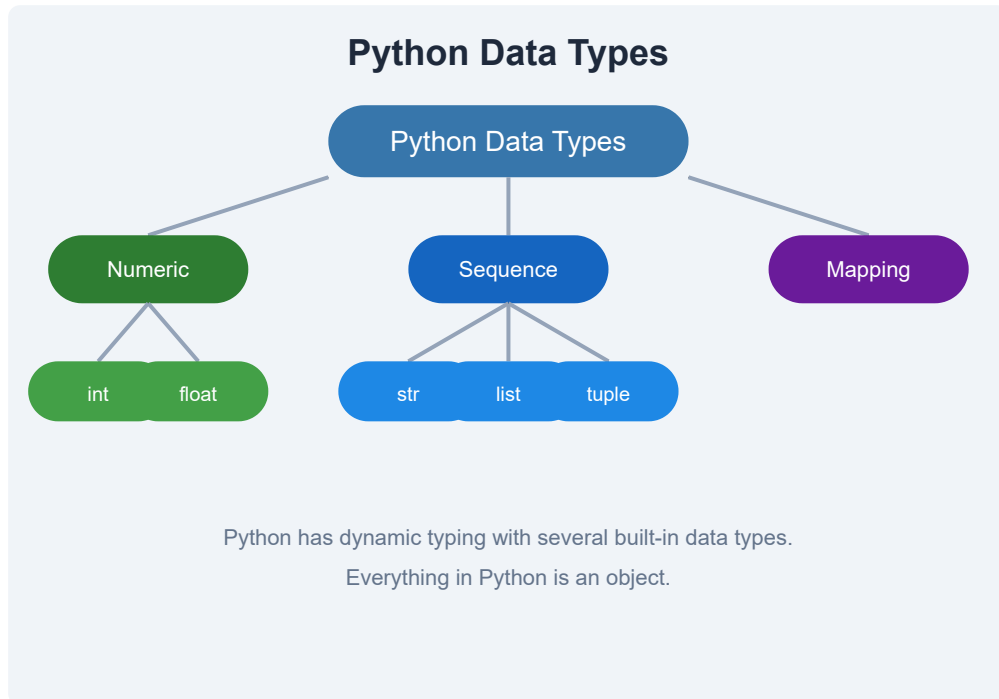
```
import argparse

parser = argparse.ArgumentParser(description="Process some integers.")
parser.add_argument("--input", required=True, help="Input file path")
parser.add_argument("--verbose", action="store_true", help="Enable verbose output")
parser.add_argument("--count", type=int, default=10, help="Number of items")

args = parser.parse_args()
print(f"Processing {args.input} with count={args.count}")
```

Exercise: Build a CLI todo-list application using argparse, SQLite for persistence, and datetime for due dates. Implement add, list, complete, and delete commands. Use JSON for export/import functionality.

Advanced Concepts



Metaclasses

A metaclass is the class of a class. Just as a class defines how instances behave, a metaclass defines how classes behave. The default metaclass is `type`.

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        dct["version"] = 1.0
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass

print(MyClass.version) # 1.0
```

You can also create classes dynamically using `type`:

```
DynamicClass = type("DynamicClass", (), {"x": 10})
obj = DynamicClass()
print(obj.x)
```

Context Managers

Context managers manage resources using the `with` statement. Implement `__enter__` and `__exit__` methods, or use `contextlib.contextmanager`.

```
class File:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

with File("test.txt", "w") as f:
    f.write("Hello, context manager!")

# Using contextlib
from contextlib import contextmanager

@contextmanager
def temporary_file(name):
    f = open(name, "w")
    try:
        yield f
    finally:
        f.close()
```

Coroutines and Async/Await

Asynchronous programming in Python uses `async def` and `await` with the `asyncio` library for concurrent I/O.

```
import asyncio

async def fetch_data(url):
    print(f"Fetching {url}...")
    await asyncio.sleep(1) # Simulate network I/O
    return f"Data from {url}"

async def main():
    urls = ["url1", "url2", "url3"]
    tasks = [fetch_data(url) for url in urls]
    results = await asyncio.gather(*tasks)
    return results

results = asyncio.run(main())
print(results)
```

Descriptors

Descriptors are objects that define `__get__`, `__set__`, or `__delete__` methods. They control attribute access on classes.

```
class PositiveNumber:
    def __get__(self, obj, objtype=None):
        return obj.__dict__.get(self.name, 0)

    def __set__(self, obj, value):
        if value < 0:
            raise ValueError("Must be positive")
        obj.__dict__[self.name] = value

    def __set_name__(self, owner, name):
        self.name = f"_{name}"

class Order:
    quantity = PositiveNumber()

order = Order()
order.quantity = 10
print(order.quantity)
```

Memory Management

Python uses reference counting and a generational garbage collector (`gc` module). Objects are freed when their reference count reaches zero.

```
import gc

gc.enable()
print(gc.get_count())

# Circular references
class Node:
    def __init__(self):
        self.ref = None

a = Node()
b = Node()
a.ref = b
b.ref = a

del a, b
gc.collect() # Force garbage collection
```

Type Hints (typing Module)

```
from typing import List, Dict, Optional, Tuple, Union

def process_items(items: List[str]) -> Dict[str, int]:
    result: Dict[str, int] = {}
    for item in items:
        result[item] = len(item)
    return result

def find_user(user_id: int) -> Optional[Dict]:
    # Returns None if not found
    pass

def combine(a: Union[int, str], b: Union[int, str]) -> str:
    return str(a) + str(b)
```

Monkey Patching

Monkey patching dynamically modifies classes or modules at runtime. Use it sparingly as it can make code harder to debug.

```
class Dog:
    def bark(self):
        return "Woof!"

    def howl(self):
        return "Howl!"

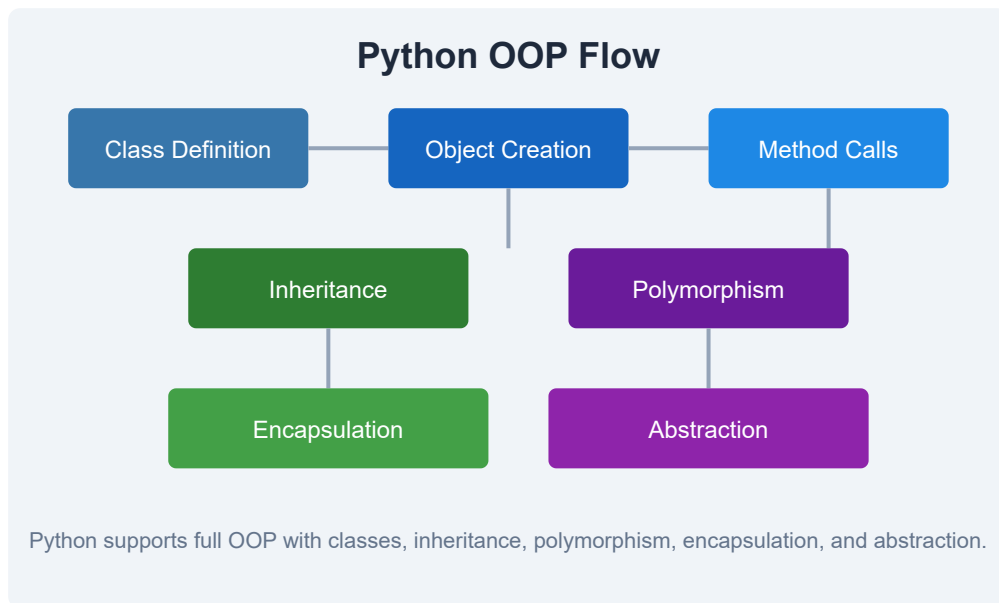
Dog.howl = howl
d = Dog()
print(d.howl()) # Howl!

# Patching a module
import math
math.original_sqrt = math.sqrt
math.sqrt = lambda x: math.original_sqrt(x) if x >= 0 else 0
```

Note: Advanced features like metaclasses and monkey patching should be used only when simpler solutions are insufficient. They add complexity and can make maintenance difficult.

Exercise: Implement a context manager that measures the execution time of a block of code and logs it. Write an async function that fetches 5 URLs concurrently using `asyncio.gather` and measures total time. Then create a custom descriptor that validates that a value is within a specified range.

Practice & Resources



Common Q&A

What is the difference between a list and a tuple?

Lists are mutable (can be changed), tuples are immutable. Lists use `[]`, tuples use `()`. Tuples are faster and can be used as dictionary keys.

What is PEP 8?

PEP 8 is Python's style guide. It recommends 4-space indentation, 79-character line limits, snake_case for functions/variables, and CamelCase for classes.

What is the difference between `deepcopy` and `shallow copy`?

Shallow copy creates a new object but inserts references to the original nested objects. Deep copy recursively copies all nested objects.

How does Python handle memory?

Python uses automatic memory management with reference counting and a generational garbage collector for cyclic references.

Top 20 Interview Questions

1. Explain the Python execution model (bytecode, interpreter, GIL).
2. What are decorators and how do they work?
3. Explain the difference between `is` and `==`.
4. How does inheritance and MRO work in Python?
5. What are generators and when should you use them?
6. Explain list comprehensions vs generator expressions.
7. What is the difference between `staticmethod`, `classmethod`, and instance methods?
8. How do you handle exceptions in Python?
9. What is the `with` statement and how does it work?
10. Explain the GIL and its impact on multithreading.
11. What are `*args` and `**kwargs`?
12. How do you manage packages in Python?
13. What are context managers?
14. Explain the difference between mutable and immutable types.
15. How does Python's garbage collection work?
16. What is the difference between `__str__` and `__repr__`?
17. Explain monkey patching.
18. What are metaclasses in Python?
19. How do you achieve concurrency in Python?
20. What is the purpose of `__init__.py`?

Quick Reference Cheatsheet

Built-in Functions

```
len(), type(), int(), str(), list(), dict(), set(), range(), enumerate(), zip()
```

Common Built-in Methods

```
# String methods
"hello".upper(), .lower(), .strip(), .split(), .join(), .replace(), .find(),

# List methods
[1,2,3].append(), .extend(), .insert(), .remove(), .pop(), .sort(), .reverse()

# Dict methods
{"a": 1}.keys(), .values(), .items(), .get(), .update(), .pop()
```

Project Ideas

1. **Calculator:** A GUI calculator using Tkinter with basic arithmetic operations.
2. **Web Scraper:** Use BeautifulSoup and requests to scrape and analyze website data.
3. **CLI Task Manager:** A command-line todo app with SQLite storage and argparse.
4. **Data Analyzer:** Load a CSV, clean it with Pandas, visualize with Matplotlib.
5. **Chat Application:** A multi-client chat server using sockets and threading.
6. **REST API:** Build a Flask or FastAPI CRUD API with JSON storage.
7. **Password Manager:** Encrypt and store passwords using cryptography library.
8. **Weather App:** Fetch weather data from a public API and display it.

Useful Resources

- [Official Python Documentation](#)
- [Real Python](#) - Tutorials and articles
- [PyPI](#) - Python Package Index
- [GitHub](#) - Open source Python projects
- [Stack Overflow](#) - Community Q&A
- [PEP Index](#) - Python Enhancement Proposals

Discussion Forums

- [r/learnpython](#) on Reddit
- Python Discord server
- [Python.org](#) mailing lists
- Dev.to Python community

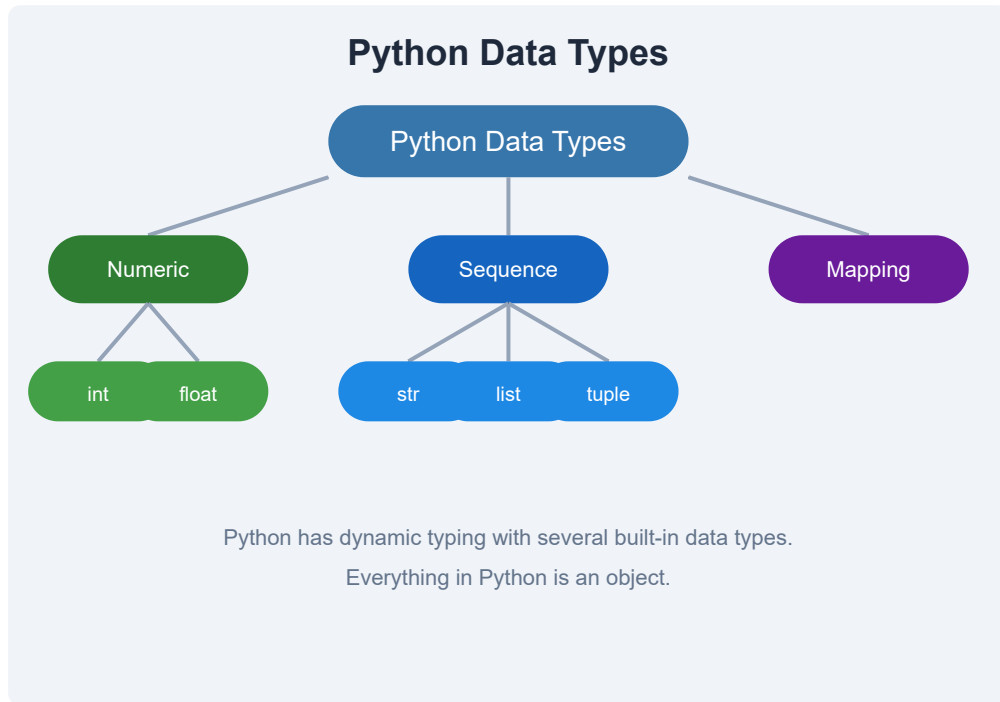
Online Quiz Suggestions

Test your knowledge with these platforms: HackerRank Python challenges, LeetCode Python problems, CodeWars katas, and the official Python tutorial quiz at docs.python.org.

Tip: Build a portfolio of 3-4 complete projects. Contribute to open source. Practice coding challenges regularly.

Final Exercise: Build a complete command-line application that reads a CSV file, processes data using Pandas, generates plots with Matplotlib, and exports a summary report as JSON. Include argument parsing, error handling, logging, and type hints.

References



Python Built-in Functions Reference

```
abs(x)           # Absolute value
all(iterable)    # True if all elements are truthy
any(iterable)    # True if any element is truthy
bin(x)           # Binary string representation
bool(x)          # Convert to boolean
chr(i)           # Unicode character from integer
dict(**kwargs)   # Create dictionary
dir([obj])       # List names in current/local scope
enumerate(iter)   # Returns (index, value) pairs
eval(expr)       # Evaluate string as Python expression
filter(func, it)  # Filter iterable with function
float(x)         # Convert to float
format(val, spec) # Format value with specification
frozenset(it)    # Create immutable set
getattr(obj, name) # Get attribute by name
globals()        # Global symbol table as dict
hasattr(obj, n)   # Check if attribute exists
hash(obj)        # Hash value of object
help([obj])      # Interactive help
hex(x)           # Hex string representation
id(obj)          # Unique identifier
input([prompt])  # Read user input
int(x)           # Convert to integer
isinstance(o, t)  # Type checking
iter(obj)        # Get iterator
len(s)           # Length of sequence
list([iter])     # Create list
locals()         # Local symbol table as dict
map(func, *iter)  # Apply function to iterables
max(iterable)    # Maximum value
min(iterable)    # Minimum value
next(iterator)   # Next item from iterator
oct(x)           # Octal string representation
open(file, mode) # Open file
ord(c)           # Unicode code point
pow(x, y)        # x**y
print(*obj)      # Print objects
range(start, stop) # Generate integer range
repr(obj)        # String representation for devs
reversed(seq)    # Reverse iterator
round(x, n)      # Round to n decimal places
```

```
set([iter])      # Create set
slice(start,stop) # Create slice object
sorted(iter)     # Return sorted list
str(obj)         # String representation
sum(iterable)    # Sum of elements
super()          # Access parent class
tuple([iter])    # Create tuple
type(obj)        # Get type of object
vars([obj])      # __dict__ of object
zip(*iterables)  # Aggregate elements from iterables
```

NumPy Reference

```
import numpy as np

# Array creation
np.array([1, 2, 3])
np.zeros((3, 4))
np.ones((2, 3, 4))
np.eye(5)           # Identity matrix
np.arange(start, stop, step)
np.linspace(0, 1, 100) # 100 evenly spaced points
np.random.rand(3, 3)   # Uniform random
np.random.randn(100)   # Normal distribution

# Array properties
arr.shape, arr.dtype, arr.size, arr.ndim, arr.T

# Reshaping
arr.reshape(2, -1)
arr.ravel()          # Flatten to 1D
np.concatenate([a, b], axis=0)
np.split(arr, 3, axis=0)

# Operations
np.add, np.subtract, np.multiply, np.divide
np.dot(a, b)          # Dot product
np.sum, np.mean, np.std, np.var, np.min, np.max, np.argmin
np.sort(arr, axis=-1)
np.where(condition, x, y)

# Linear algebra
np.linalg.inv, np.linalg.det, np.linalg.eig
np.linalg.svd, np.linalg.qr
```


Pandas Reference

```
import pandas as pd

# Series
s = pd.Series([1, 2, 3], index=["a", "b", "c"])
s.values, s.index, s.name

# DataFrame
df = pd.DataFrame(data, index=rows, columns=cols)
df.head(n), df.tail(n), df.info(), df.describe()
df.columns, df.index, df.dtypes, df.shape

# Selection
df["col"]           # Single column
df[["a", "b"]]      # Multiple columns
df.loc["row"]       # Label-based
df.iloc[0]          # Position-based
df[df["age"] > 30]   # Boolean indexing

# Cleaning
df.dropna(), df.fillna(val), df.drop_duplicates()
df.rename(columns={}), df.replace(old, new)

# Grouping and aggregation
df.groupby("col").mean()
df.groupby("col").agg({"salary": "sum", "age": "mean"})
df.pivot_table(values="val", index="row", columns="col")

# Merge and join
pd.merge(df1, df2, on="key", how="inner")
df1.join(df2, how="left")

# Input/output
pd.read_csv, pd.read_excel, pd.read_json, pd.read_sql
df.to_csv, df.to_excel, df.to_json, df.to_sql
```

Matplotlib Reference

```
import matplotlib.pyplot as plt

# Plot types
plt.plot(x, y, fmt)           # Line plot
plt.scatter(x, y)             # Scatter plot
plt.bar(x, height)            # Bar chart
plt.barh(y, width)            # Horizontal bar
plt.hist(data, bins)          # Histogram
plt.boxplot(data)             # Box plot
plt.pie(sizes, labels)         # Pie chart
plt.imshow(image)             # Image display
plt.contour(X, Y, Z)          # Contour plot

# Customization
plt.title("Title", fontsize=14)
plt.xlabel("X label"), plt.ylabel("Y label")
plt.xlim(0, 10), plt.ylim(0, 100)
plt.xticks(rotation=45)
plt.grid(True, alpha=0.3)
plt.legend(loc="best")
plt.tight_layout()

# Colors and styles
plt.plot(x, y, color="red", linestyle="--", linewidth=2, marker="o")
plt.style.use("ggplot")

# Subplots
fig, axes = plt.subplots(2, 3, figsize=(10, 6))
axes[0, 0].plot(x, y)
plt.tight_layout()

# Saving
plt.savefig("plot.png", dpi=300, bbox_inches="tight")
plt.show()
```

SciPy Reference

```
from scipy import optimize, stats, signal, integrate, linalg

# optimize
optimize.minimize(fun, x0, method="BFGS")
optimize.curve_fit(func, xdata, ydata)
optimize.root(func, x0)
optimize.linear_sum_assignment(cost_matrix)

# stats
stats.norm.pdf(x, loc=0, scale=1)      # PDF
stats.norm.cdf(x)                      # CDF
stats.norm.ppf(q)                      # Percent point function
stats.ttest_ind(group1, group2)        # Independent t-test
stats.ttest_rel(before, after)         # Paired t-test
stats.chisquare(f_obs, f_exp)
stats.pearsonr(x, y)                   # Correlation
stats.describe(data)                   # Summary statistics
stats.zscore(data)                     # Standardize

# signal
signal.convolve(signal, kernel)
signal.correlate(a, v)
signal.spectrogram(x)
signal.savgol_filter(x, window_length, polyorder)

# integrate
integrate.quad(func, a, b)              # 1D integration
integrate.dblquad(func, a, b, gfun, hfun)
integrate.solve_ivp(fun, t_span, y0)   # ODE solver
```

Note: These reference pages summarize the most commonly used functions. For full documentation, visit docs.python.org, numpy.org/doc, pandas.pydata.org, matplotlib.org, and docs.scipy.org.

Exercise: Use this reference sheet to solve a real problem: load a dataset using Pandas, filter and group it, create two subplot visualizations, run a statistical test using SciPy, and save the results. Refer back to the function signatures above as needed.